MINOR PROJECT SYNOPSIS (PR 1103)

CureNet: Health Intelligence for Fragmented Healthcare Systems

Bridging Data Silos to Enable Patient-centric Integrated care across India

PREPARED BY

Labish Bardiya (2023BTech106)
Rakshika Sharma (2023BTech065)

FACULTY SUPERVISOR

Mr. Gaurav Raj

Department of Computer Science Engineering
Institute of Engineering and Technology
JK Lakshmipat University, Jaipur

January, 2026

Abstract

India's healthcare system faces significant inefficiencies due to fragmented medical records, with studies indicating that approximately 65% of hospitals lack electronic medical record systems. This fragmentation contributes to nearly 32% duplicate diagnostic testing and limits effective care delivery in public hospitals where average consultation time is approximately two minutes. CureNet proposes an AI-assisted health intelligence platform that unifies patient medical records on top of the Ayushman Bharat Digital Mission (ABDM) infrastructure, which has issued over 770 million ABHA IDs.

The proposed system provides instant clinical summaries, multilingual patient-friendly explanations in Indian languages such as Hindi, Tamil, and Telugu, and rapid emergency health snapshots for critical situations. Using transformer-based natural language processing models and FHIR-aligned data structures, CureNet aims to demonstrate technical feasibility through a three-month prototype. The project focuses on tier-2 and tier-3 cities, which serve nearly 67% of India's population and face the greatest information gaps. The outcome is a deployable MVP that validates how integrated health information systems can improve consultation efficiency, reduce duplication, and enhance patient safety.

1. Introduction

1.1 The Crisis of Medical Fragmentation in India

India's healthcare ecosystem comprises government hospitals, private clinics, diagnostic centres, and rural health facilities that largely operate as isolated information systems. Patient data such as diagnoses, medication history, allergies, and investigation reports remain scattered across institutions and paper records. As a result, clinicians often make decisions without access to complete medical context. Evidence shows that nearly one-third of transferred patients undergo duplicate diagnostic testing, with a substantial portion being clinically unnecessary.

1.2 Tier-2 and Tier-3 Cities: Where the Crisis is Acute

While metropolitan cities have seen partial adoption of health IT systems, tier-2 and tier-3 cities. home to approximately 67% of India's population. continue to rely heavily on manual and paper-based processes. Doctor-to-patient ratios in these regions can reach 1:25,000, and many Primary Health Centres face unreliable connectivity and limited infrastructure. Although ABDM has established digital identity and record-linking at scale, real-time access and usability at the point of care remain limited in these settings.

1.3 The Time Efficiency Crisis

Government hospitals in India operate under extreme time constraints, with average consultation durations of about two minutes. In such conditions, clinicians spend a significant portion of time collecting patient history rather than performing clinical assessment or counseling. Missing or inaccessible records directly increase the risk of diagnostic errors and inappropriate treatment. CureNet addresses this by presenting structured summaries of key patient information at registration and consultation.

1.4 Language Barriers as a Patient-Safety Issue

India's linguistic diversity presents a major challenge in healthcare delivery. Millions of patients interact with providers using a non-shared language, leading to misunderstandings of diagnoses, prescriptions, and follow-up instructions. Existing digital health systems rarely support multilingual patient education. CureNet incorporates multilingual NLP to generate patient-friendly explanations and alerts in major Indian languages, reducing communication-related risks.

2. Objectives

Unify Fragmented Medical Records
Develop a platform that aggregates patient data from government hospitals, private clinics, and diagnostic centres into a single, longitudinal digital record accessible at the point of care.

Improve Consultation Quality Within 2 Minutes
Provide instant AI-generated clinical summaries so government doctors can use limited consultation time for diagnosis and counseling instead of repetitive history-taking.

Reduce Duplicate Tests and Medication Errors
Enable clinicians to view prior investigations and complete medication histories, targeting measurable reductions in duplicate diagnostics and preventable drug-related adverse events.

Support Tier-2 and Tier-3 Deployments
Design CureNet to work reliably in low-resource settings, with intermittent internet, limited hardware, and high patient loads, where the impact of better information flow is greatest.

Break Language Barriers in Care
Use multilingual NLP to generate patient-friendly explanations and instructions in major Indian languages, improving understanding and adherence for non-English-speaking patients.

Enable Safe Emergency Access
Provide emergency snapshots (allergies, medications, critical conditions) that can be accessed securely and rapidly by ambulance and casualty teams during life-threatening events.

Ensure Privacy, Consent, and Compliance
Implement consent-driven access control aligned with ABDM, IT Act 2000, and proposed DISHA requirements, giving patients transparency and control over who sees their data.

3. Key Features

3.1 Unified Longitudinal Health Record

- Aggregates data from government hospitals, private clinics, and diagnostic centres into a single patient timeline.

3.2 AI-Generated Clinical Summaries

- Generates concise summaries of diagnoses, medications, allergies, and recent investigations for rapid clinical review.

3.3 Multilingual Patient Explanations

- Converts complex medical content into simplified explanations in Indian languages, with optional audio support.

3.4 Emergency Health Snapshot

- Provides rapid access to critical information such as allergies and active medications during emergencies.

3.5 Consent and Privacy Control

- Allows patients to approve or revoke provider access, maintaining transparent access logs.

3.6 Provider Search and Query Interface

- Supports quick retrieval of relevant medical information using structured queries.

3.7 Offline-First Tier-2/3 Design

- Caches essential data locally to support care delivery in low-connectivity environments.

3.8 Smart Medication Reminders & Adherence Tracking

- Sends multilingual reminders for prescribed medications and follow-up actions.

4. Technology Stack

4.1 Frontend

- **Web App:** React.js with TypeScript for type-safe, component-based development.
- **Mobile App:** React Native for Android/iOS, designed as offline-first for tier-2/3 and rural usage.
- **UI Library:** Material-UI / NativeBase for consistent design and fast prototyping.
- **State Management:** Redux or React Query for managing API data and caching.

4.2 Backend

- **Framework:** Node.js with Express.js (or NestJS) for REST/GraphQL APIs.
  API Design: FHIR-compatible endpoints for interoperability with ABDM and hospital systems.
- **Authentication:** JWT + OAuth2/OIDC to integrate with ABHA / ABDM identity and role-based access control.

4.3 Database & Storage

- **Primary DB:** PostgreSQL for relational data (patients, visits, consents, alerts).
- **Search:** Elasticsearch for fast text/semantic search over clinical notes.
- **Cache:** Redis for emergency snapshots, session data, and frequently accessed summaries.
  File Storage: Object storage (e.g., AWS S3 or on-prem equivalent) for PDFs, images, and scanned records.

4.4 NLP & AI Services

- **Models:** HuggingFace Transformers (BART/T5 for summarization, mT5/IndicTrans for multilingual text).
  NLP Stack: Python microservices using FastAPI, spaCy/Flair for medical entity extraction.

4.5 Deployment & DevOps

- **Containers:** Docker for all services.
- **Orchestration:** Kubernetes (or Docker Compose for prototype) with horizontal scaling.
- **CI/CD:** GitHub Actions / GitLab CI for automated build, test, and deployment.
- **Hosting:** Cloud provider with India region (e.g., AWS Mumbai) to meet data-residency requirements.

4.6 Security & Compliance

- **Encryption:** HTTPS/TLS for data in transit; AES-256 for data at rest.
- **Audit Logs:** Central logging (ELK/EFK stack) for all access and consent events.
- **Compliance Orientation:** Designed to align with ABDM guidelines, IT Act 2000, and proposed DISHA privacy principles.

5. Scalability & Optimization Approach

5.1 Data & Database Layer

- **Patient-centric sharding:** Partition records by patient/region (e.g., state or district) so a surge in one city does not impact the whole system. This suits CureNet because most queries are "all data for this patient" or "patients in this facility".
- **Hybrid storage model:** Keep structured clinical metadata (IDs, dates, facility, key flags) in PostgreSQL and large documents (PDFs, images) in object storage. This reduces DB load and speeds queries that only need summaries.
- **Pre-computed views:** Maintain denormalized "summary tables" (latest vitals, active meds, last 5 reports) that can be read in a single query during registration or ED triage, avoiding expensive joins over the full longitudinal history.

5.2 Caching & Read Performance

- **Hot-path caching:** Cache emergency snapshots, latest summaries, and consent decisions in Redis so ambulance and casualty screens load in <1 second. Most emergency lookups hit this small cached set, not the full database.
- **ABDM gateway caching:** For ABDM lookups that are slow or rate-limited, cache responses with short TTL (e.g., 5-15 minutes) so multiple clinicians viewing the same patient do not repeatedly hit ABDM APIs.
- **Edge and device caching:** On mobiles used in tier-2/3 PHCs, cache the last N patients seen and their summaries locally with encrypted storage, enabling quick re-access even if the network drops.

5.3 Network & I/O Optimization for Tier-2/3 India

- **Low-bandwidth payloads:** Design "summary-first" APIs returning compact JSON (IDs, text, small thumbnails) and load full PDFs/images only on demand, which is crucial on 2G/3G or congested hospital Wi-Fi.
- **Delta sync instead of full sync:** When a device reconnects, sync only new/changed records since the last timestamp, reducing data transfer for rural deployments.
- **Asynchronous jobs for heavy tasks:** OCR of scanned records, bulk imports from hospital HIS, and nightly analytics run as background workers (e.g., using a job queue) so they never block clinical workflows.

5.4 NLP/AI Performance & Cost Control

- **Tiered model strategy:**
  - Lightweight distilled models for common, latency-sensitive paths (visit summary at registration).
  - Full-size models for less frequent, offline jobs (deep discharge summary, complex translation), possibly batch-processed overnight.
- **Result reuse:** Store generated summaries and translations with a version hash of the source document; only re-run NLP when the document actually changes. This avoids repeated inference for the same discharge summary seen by many clinicians.

5.5 Horizontal Scalability & Reliability

- **Service decomposition:** Separate services for auth/consent, patient record management, NLP, notifications, and analytics so each can scale independently based on load (e.g., notifications spike in evenings, NLP during working hours).
- **Stateless APIs:** Keep application servers stateless with all session and cache data in Redis or a similar store; this allows simple horizontal scaling behind a load balancer.
- **Graceful degradation:** If NLP or ABDM services are temporarily unavailable, the system should still show last known summaries and local records, flagging them as "not up to date" rather than failing entirely, critical for emergencies.

5.6 Observability & Continuous Optimization

- **Fine-grained metrics:** Track per-endpoint latency, error rate, and usage (e.g., time to load summary, time to fetch ABDM data, cache hit ratio) so bottlenecks in registration or ED flows are visible.
- **Real-world load testing:** Simulates OPD patterns (hundreds of short bursts around registration and morning clinics) rather than uniform load, reflecting actual Indian hospital usage.
- **Feedback-driven tuning:** Use provider feedback from pilot sites (e.g., "summary screen is slow at 9-11 AM") to prioritize index changes, cache rules, or model optimizations that directly improve clinical experience.

6. Methodology

6.1 Requirement Analysis and Research (Week 1)

- Visit or interview doctors, nurses, and admins in at least one government hospital, one private hospital, and one diagnostic centre to map how patient data currently flows and where it breaks.
- Collect sample de-identified prescriptions, lab reports, and discharge summaries to understand document formats and languages.

- Study ABDM/ABHA guidelines and existing APIs to decide what can realistically be integrated in a prototype within 3 months.

6.2 System Design (Week 2)

- Design high-level architecture: mobile/web clients → API gateway → core services (records, consent, NLP, notifications) → database and cache.
- Model core entities (Patient, Visit, Summary, EmergencyProfile, Consent, Notification) and define how they map to FHIR resources for future ABDM integration.
- Sketch user flows for three main journeys: OPD registration + doctor view, patient viewing summary and reminders, and emergency access by paramedic.

6.3 Modular Development (Weeks 3-8)

- Module 1- Record Ingestion & Storage: Build APIs and UI to add/edit patient details, upload basic records, and store them as structured entries plus files.
- Module 2 - Clinical Summary Engine: Implement summarization pipeline using one transformer model (e.g., BART/T5) to create concise, structured summaries from visit data and notes.
- Module 3 - Multilingual Explanations & Reminders: Implement Hindi (and optionally one more language) explanation generator for lab values and medicine instructions, plus notification scheduler for dosage and follow-ups.
- Module 4 - Emergency Snapshot: Build a minimal view (allergies, chronic conditions, current drugs, emergency contact) accessible via patient ID/QR with strict access checks.
- Module 5 - Consent & Access Control: Create basic patient-side screen to approve/revoke provider access and log every access event.

6.4 Integration and Testing (Weeks 9-10)

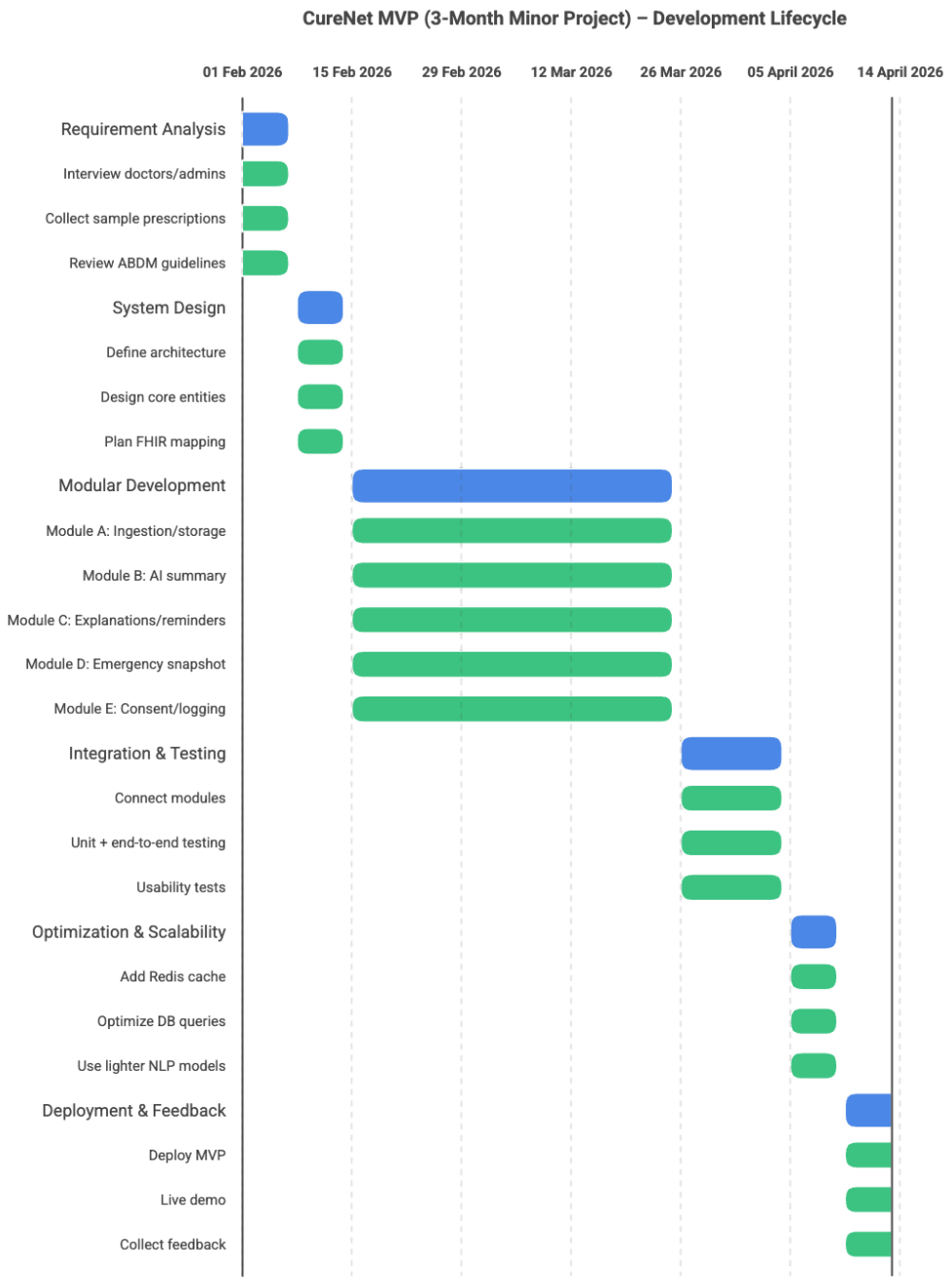
- Integrate all modules into a single workflow: registration → summary generation → doctor view → patient explanation/reminders → emergency access.
- Perform unit testing for each module and end-to-end testing for the 3 core flows (OPD, patient self-view, emergency).
- Conduct small usability tests with 3-5 clinicians and 3-5 lay users to validate that summaries, explanations, and reminders are understandable and fast enough.

6.5 Optimization and Scalability Enhancements (Week 11)

- Profile response times for summary generation, emergency snapshot loading, and reminder scheduling; add caching where needed (e.g., Redis for summaries and emergency profiles).
- Optimize database queries (indexes, pagination) and reduce payload sizes for mobile clients, especially on simulated low-bandwidth networks.
- Replace any slow NLP step with a lighter/quantized model or pre-compute results where possible.

6.6 Deployment and Feedback (Week 12)

- Deploy the MVP on a cloud instance (India region) with HTTPS, basic monitoring, and backup policies.
- Prepare demo scenarios and collect structured feedback from faculty and, if feasible, one real clinical stakeholder (doctor or hospital IT person).
- Document limitations, scalability options (ABDM integration, multi-hospital support), and a roadmap for extending from prototype to production-grade system.



8. Expected Outcomes

8.1 Prototype-Level Outcomes (End of 3 Months)

1. **Working MVP for Fragmented Records**

    - CureNet prototype successfully stores and retrieves longitudinal records (visits, prescriptions, lab results), demonstrating that fragmented data from different facilities can be unified into a single timeline.

2. **Instant Clinical Summary View**

    - Doctor-facing summary screen loads in **minimal time** for a typical patient, allowing faculty/clinician reviewers to see diagnoses, current medicines, allergies, and last few investigations without opening multiple documents.

3. **Multilingual Patient Explanation & Reminders**

    - The system generates clear medication and lab-report explanations in **English and at least one Indian language (e.g., Hindi)** and triggers dosage reminders according to test prescriptions for at least three common chronic conditions.

4. **Emergency Snapshot Functionality**

    - Emergency view showing allergies, key conditions, and current drugs is accessible from patient ID/QR on demo data, even on simulated low-bandwidth networks.

9. Future Enhancements

9.1 Product-Level Enhancements

1. **Hardware-Integrated Self Check-In (Fingerprint + Aadhaar)**

    - Design a low-cost kiosk with a **fingerprint scanner linked to Aadhaar** so patients can authenticate themselves without a receptionist.
    - On successful match, the kiosk auto-fetches the patient's CureNet profile and **prints or displays a digital token/receipt** containing visit ID, queue number, and a QR code that doctors can scan to load the full longitudinal record.
    - This reduces congestion at the registration desk, minimizes manual data entry errors, and standardizes check-in across government hospitals, private clinics, and diagnostic centres.

2. **Full ABDM Integration & Cross-Facility Record Routing**

    - Replace mock FHIR endpoints with live ABDM connectors so CureNet can **push and pull records from any ABDM-compliant hospital** in India.
    - Implement intelligent routing so that when a patient checks in (by Aadhaar/fingerprint/ABHA/QR), the system automatically discovers and fetches records from all linked facilities, not only the local hospital.

3. **Richer Clinical Modules**

    - Add structured templates for chronic diseases (diabetes, hypertension, COPD, CKD) so longitudinal views include disease-specific metrics (HbA1c trend, BP readings, eGFR).
    - Integrate simple clinical decision support (e.g., drug-drug interaction alerts, renal dose warnings) on top of the unified medication list.

4. **Advanced Patient Engagement Tools**

    - Expand reminders beyond medicines to include **follow-up appointments, lab re-tests, and vaccination schedules**.
    - Provide interactive "explainers" with visuals for common conditions and procedures in multiple Indian languages, tailored to literacy level.

10. Conclusion

CureNet is conceived as a focused, technically realistic response to one of India's most damaging healthcare gaps: the fragmentation of patient information across government hospitals, private clinics, and diagnostic centres, especially in tier-2 and tier-3 cities. By designing an MVP that can unify basic records, generate instant doctor-facing summaries, produce multilingual patient explanations, send medicine reminders, and expose a safety-critical emergency snapshot, this minor project demonstrates how even a small, well-architected system can meaningfully support 2-minute consultations and overburdened public facilities. The chosen stack, React/React Native, Node.js, PostgreSQL, Redis, and transformer-based NLP, shows that modern, open technologies can be combined with ABDM/FHIR principles to create a foundation that is both implementable in 3 months and extensible to real hospital environments later.